

SIMATS ENGINEERING

ASSESSMENT TOOL 2

**COURSE CODE: CSA1407
DESIGN**

COURSE NAME: COMPILER

COURSE OUTCOME COVERED

C03: *Analyze syntax-directed translation method using synthesized and inherited attributes to generate a Parser Tree. (BL4)*

Assessment Tool Weightage: 20%

QUESTIONS: 5

Total Marks:100

Annexure A

Q. No	Understanding of Concepts (25)	Experimental Design (25)	Use of Tools (20)	Analysis of Results (20)	Documentation (10)	Total Score (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
OVERALL RUBRICS VALUE						

	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 25	/ 25	/ 20	/ 20	/ 10	/ 100

CASE STUDY

Code of Conduct:

I Rekha S K (192511023) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Rekha S K

Annexure A

CASE STUDY

Set 4

1.

A game development company is creating a custom scripting language for controlling character movements and animations. The compiler team uses syntax-directed definitions to generate syntax trees for arithmetic and logical expressions during parsing. Initially, bottom-up evaluation with S-attributed definitions works well for simple

expressions, but difficulties arise when inherited attributes are needed for function calls and nested scopes. During semantic analysis, the compiler detects several type mismatches between integer, float, and boolean variables. To ensure smooth execution, the developers implement a type checker with automatic type conversion support.

a) How do syntax-directed definitions assist in syntax tree construction?

b) Why are type conversions necessary in semantic analysis?

2

A banking software firm is designing a secure programming language for financial transactions where semantic correctness is critical. The compiler translates expressions into syntax trees using top-down translation techniques, but complex nested statements cause problems in attribute evaluation. To improve efficiency, the team shifts to bottom-up evaluation of L-attributed definitions for handling variable declarations and parameter passing. During testing, errors occur because equivalent type expressions are not properly verified across modules. The compiler team then integrates a type system capable of checking compatibility and preventing unsafe assignments.

a) Compare top-down and bottom-up evaluation techniques.

b) How does type equivalence improve program reliability?

3

In a compiler research project, students are developing a language that supports arrays, functions, and conditional statements. The parser successfully generates syntax trees, but semantic actions fail when inherited and synthesized attributes are mixed incorrectly. To resolve this issue, the students redesign the syntax-directed definitions using both S-attributed and L-attributed approaches. During semantic checking, the compiler identifies invalid operations involving incompatible type expressions such as arrays and scalar variables. The students therefore implement a simple type checker with explicit type conversion rules.

- a) What challenges arise in evaluating inherited and synthesized attributes?
- b) Explain the role of type checking in semantic analysis.

4

A software engineering team is developing a compiler for an IoT programming language where commands from sensors must be translated efficiently into intermediate code. The compiler initially uses syntax-directed translation with topdown parsing, but recursive dependencies in semantic rules make translation difficult. To improve semantic processing, the team adopts bottom-up evaluation and constructs syntax trees for all arithmetic and logical expressions. During execution, incorrect results occur because variables with different type expressions are combined without validation. To solve this issue, the compiler introduces type equivalence checking and automatic type conversions.

- a) Why are syntax trees important in syntax-directed translation?
- b) How do type equivalence and conversion improve compiler accuracy?

5

A university compiler lab is developing a mini language interpreter for mathematical computations. The syntax analyzer generates parse trees successfully, but semantic analysis fails because attribute values are not propagated correctly across grammar productions. To address this issue, the students implement syntax-directed definitions and compare S-attributed and L-attributed evaluation methods. As the language expands to support floating-point and integer operations together, the compiler frequently encounters type mismatch errors. The team then designs a type system capable of checking compatibility and performing safe implicit conversions during translation.

- a) Differentiate between S-attributed and L-attributed definitions.
- b) Why is a type system necessary in compiler design?

SET-4

Answers

1. Game Development Scripting Language

a) How do syntax-directed definitions assist in syntax tree construction?

Syntax-Directed Definitions (SDDs) associate attributes and semantic rules with grammar productions. They help the compiler construct a syntax tree while parsing the source program. When an expression is parsed, each grammar production is given semantic rules that specify how the corresponding nodes of the syntax tree should be created. Synthesized attributes can be used to pass information from child nodes to their parent nodes.

For example, consider:

$$E \rightarrow E + T$$

A semantic rule can construct a tree node representing +, with the syntax trees of E and T as its children.

Thus, syntax-directed definitions help to:

- Associate semantic information with grammar rules.
- Construct syntax trees systematically during parsing.
- Represent arithmetic and logical expressions hierarchically.
- Store operators as internal nodes and operands as leaf nodes.
- Support later compiler phases such as semantic analysis and intermediate code generation.

Example:

For the expression:

$$a + b * c$$

the syntax tree represents * below +, correctly reflecting operator precedence.

$$\begin{array}{c} + \\ / \backslash \\ a \quad * \\ / \backslash \\ b \quad c \end{array}$$

Therefore, SDDs provide a systematic way of constructing syntax trees from the grammar during parsing.

b) Why are type conversions necessary in semantic analysis?

Type conversion is necessary when operands or expressions have different data types but need to be used together in an operation.

For example:

```
int a;
```

```
float b;
```

```
a + b;
```

Here, a is an integer and b is a floating-point value. The compiler must convert one type into another before performing the operation.

Type conversions help to:

- Resolve type mismatches between operands.
- Allow operations involving compatible data types.
- Prevent invalid assignments and expressions.
- Maintain semantic correctness.
- Avoid unexpected execution errors.
- Support mixed-type expressions such as integer + float.

There are two common types:

1. Implicit conversion:

The compiler automatically converts one type into another.

```
int a = 10;
```

```
float b = 2.5;
```

```
float c = a + b;
```

Here, a may be automatically converted from int to float.

2. Explicit conversion:

The programmer specifies the conversion.

```
float x = 5.7;
```

```
int y = (int)x;
```

Thus, type conversion allows compatible values of different types to work together safely during program execution.

2. Banking Software Language

a) Compare top-down and bottom-up evaluation techniques.

Top-down and bottom-up evaluation describe how attributes and semantic information can be evaluated while processing a syntax tree.

Feature	Top-Down Evaluation	Bottom-Up Evaluation
Direction	Root → leaves	Leaves → root
Common association	Inherited attributes	Synthesized attributes
Processing	Starts from the start symbol	Starts from input/leaf nodes
Parser relationship	Commonly associated with LL/top-down parsing	Commonly associated with LR/bottom-up parsing
Information flow	Parent/sibling information can be passed downward	Child information is propagated upward
Advantage	Useful when context must be passed to lower nodes	Efficient for synthesized information
Example use	Variable declarations, scope/context information	Expression evaluation and syntax-tree construction

Top-down evaluation:

Begins at the root of the syntax tree and proceeds toward the leaves. It is useful when inherited attributes are required, because information such as type, scope, or context can be passed from parent nodes to child nodes.

Bottom-up evaluation:

Begins with the leaves and moves toward the root. It is especially suitable for S-attributed definitions, where attributes are synthesized from children to their parents.

Therefore, bottom-up evaluation is generally convenient for LR parsing and synthesized attributes, while top-down evaluation is useful when contextual or inherited information must be propagated downward.

b) How does type equivalence improve program reliability?

Type equivalence determines whether two type expressions represent the same or compatible type.

For example:

int

float

array of int

array of float

are different types. The compiler must determine whether they can be used interchangeably.

Type equivalence improves reliability by:

- Detecting incompatible assignments.
- Preventing invalid operations.
- Ensuring function arguments match parameter types.
- Detecting incorrect return types.
- Preventing unsafe data conversions.
- Maintaining consistency between different program modules.
- Detecting errors during compilation rather than execution.

For example:

int x;

boolean y;

x = y;

The compiler detects that int and boolean are not equivalent and reports a type error.

Thus, type equivalence provides compile-time safety and consistency, reducing the possibility of runtime errors.

3. Arrays, Functions and Conditional Statements

a) What challenges arise in evaluating inherited and synthesized attributes?

The main challenge is that inherited and synthesized attributes have different directions of information flow.

- Synthesized attributes obtain information from child nodes and pass it upward.
- Inherited attributes obtain information from the parent or siblings and pass it downward or sideways.

Several problems can occur when both are used incorrectly:

1. Dependency problems:

One attribute may depend on another attribute that has not yet been evaluated.

2. Incorrect evaluation order:

Attributes must be evaluated in an order that satisfies their dependencies.

3. Circular dependencies:

An attribute may indirectly depend on itself, making evaluation impossible.

4. Context propagation problems:

Information such as type, scope, or parameter information may not reach the correct child node.

5. Parser restrictions:

Some evaluation methods are easier to implement with certain parsing techniques.

For example, an inherited attribute may need information from a parent before a child can be evaluated, while the parent's synthesized attribute may depend on that same child. This can create a dependency cycle.

Therefore, the compiler must carefully design semantic rules and evaluation order to ensure that all attribute dependencies can be resolved.

b) Explain the role of type checking in semantic analysis.

Type checking is the process of verifying that the data types used in a program are compatible with the operations being performed.

It is an important part of semantic analysis.

For example:

`int a;`

`float b;`

`boolean c;`

`a + b;    // Valid`

`a + c;    // Invalid`

The type checker verifies whether each operation is valid according to the language's type rules.

Its major roles include:

- Detecting type mismatches.
- Checking assignment compatibility.

- Checking operands of arithmetic and logical operations.
- Checking function parameter and argument types.
- Checking function return types.
- Validating array indexing operations.
- Preventing invalid operations between arrays and scalar values.
- Applying permitted type conversions.

For example:

```
int x;
```

```
boolean y;
```

```
x = y;
```

The compiler reports a type error because an integer variable cannot directly receive a Boolean value.

Thus, type checking ensures semantic correctness, type safety, and reliable program execution.

4. IoT Programming Language

a) Why are syntax trees important in syntax-directed translation?

A syntax tree is a hierarchical representation of the structure of a program or expression. It removes unnecessary grammar details and represents the essential structure required for semantic processing.

Syntax trees are important because they:

- Represent the hierarchical structure of expressions.
- Clearly show operator precedence and associativity.
- Provide a convenient structure for semantic analysis.
- Help evaluate arithmetic and logical expressions.
- Support type checking.
- Make intermediate code generation easier.
- Allow optimization of expressions.
- Help translate source programs into intermediate representation.

For example:

$a + b * c$

is represented as:

```
      +
     /\
    a  *
     /\
    b  c
```

This clearly shows that $b * c$ is evaluated before $a +$.

In syntax-directed translation, semantic actions can operate on the nodes of this tree to generate intermediate code.

Therefore, syntax trees act as an important intermediate representation between parsing, semantic analysis, and code generation.

b) How do type equivalence and conversion improve compiler accuracy?

Type equivalence checks whether two types are considered the same or compatible according to the language's type rules.

Type conversion changes a value from one compatible type to another.

For example:

`int a = 10;`

`float b = 2.5;`

`b = a + b;`

The compiler can convert `a` from `int` to `float` before performing the addition.

Together, type equivalence and conversion improve compiler accuracy by:

- Detecting incompatible operations.
- Preventing invalid assignments.
- Ensuring operands have compatible types.
- Supporting safe mixed-type expressions.
- Reducing runtime errors.
- Maintaining consistency between modules and functions.

- Preventing incorrect interpretation of data.
- Allowing automatic conversion when it is safe.

For example:

int x;

boolean y;

x = y;

This should produce a type error because int and boolean are not compatible.

Thus, type equivalence detects whether types are compatible, while type conversion allows compatible values to be safely used together.

5. Mini Language Interpreter

a) Differentiate between S-attributed and L-attributed definitions.

S-attributed and L-attributed definitions are two types of syntax-directed definitions based on how attributes are used.

Feature	S-Attributed Definition	L-Attributed Definition
Attributes used	Only synthesized attributes	Synthesized and restricted inherited attributes
Information flow	Child $\rightarrow$ parent	Parent/left siblings $\rightarrow$ child and child $\rightarrow$ parent
Evaluation direction	Mainly bottom-up	Can be evaluated left-to-right
Suitable parsing	Bottom-up/LR parsing	Top-down and suitable bottom-up implementations
Complexity	Simple	More flexible
Common use	Expression evaluation, syntax-tree construction	Type declarations, parameter passing, scope/context information

S-attributed definition

An S-attributed definition uses only synthesized attributes.

Information flows from the children to their parent.

Example:

$E \rightarrow E1 + T$

The value of E can be calculated from E1 and T.

Therefore, S-attributed definitions are naturally suitable for bottom-up evaluation.

L-attributed definition

An L-attributed definition allows both synthesized attributes and certain inherited attributes.

An inherited attribute of a symbol can depend on:

- Attributes of its parent.
- Attributes of symbols to its left.

This makes L-attributed definitions useful for passing contextual information such as types, scopes, and parameter information.

Therefore, S-attributed definitions are simpler and use only synthesized attributes, whereas L-attributed definitions are more general and support controlled inherited attributes.

b) Why is a type system necessary in compiler design?

A type system defines the rules that determine how different data types can be declared, combined, assigned, and converted in a programming language.

It is necessary because it helps the compiler detect semantic errors before program execution.

For example:

```
int x;
```

```
boolean y;
```

```
x = y;
```

The type system identifies this as an invalid assignment.

A type system helps to:

1. Detect type mismatches
Prevents incompatible types from being used together.
2. Check assignments
Ensures that the value assigned to a variable is compatible with its declared type.
3. Check expressions
Ensures that operators are applied to valid operands.
4. Check function calls
Verifies that arguments match parameter types.
5. Support type conversion
Allows safe conversion between compatible types such as int and float.

6. Improve program reliability

Detects errors during compilation instead of allowing them to cause unexpected runtime behavior.

7. Improve code generation

The compiler knows the size, representation, and operations associated with each type.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, wellstructured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand